

Algoritmos y Estructuras de Datos

Arreglos Bidimensionales



Contenidos del documento:

- ❖ Rectangulares
- ❖ Recorridos
- ❖ Matrices Especiales (cuadradas), triangulares, diagonal y diagonal principal
- ❖ Operaciones Típicas

Matrices Rectangulares:

Una matriz rectangular es aquella donde **todas las filas tienen exactamente la misma cantidad de columnas**.

Declaracion e Inicializacion:

Para declarar una matriz se usan comas , dentro de los corchetes. El primer espacio define las filas y el segundo las columnas [**filas, columnas**].

```
// Declaración de una matriz de 3 filas y 4 columnas (vacía, rellena de 0)
int[,] matriz = new int[3, 4];

// Declaración con inicialización explícita (Matriz de 2x3)
int[,] matrizDatos = {
    { 1, 2, 3 }, // Fila 0
    { 4, 5, 6 }  // Fila 1
};
```

A diferencia de con los vectores, no usen `matriz.Length` para los bucles, ya que esto te dará el total absoluto de elementos (en la matriz de 2x3 devolvería 6). En su lugar, usamos `.GetLength()`:

- `matriz.GetLength(0)`: Devuelve la cantidad de filas.
- `matriz.GetLength(1)`: Devuelve la cantidad de columnas.

Recorridos:

Dependiendo de cómo aniden los ciclos `for`, pueden procesar la información en dos direcciones distintas.

Recorrido Fila - Columna (Por filas)

Es el más común. Se lee la matriz de izquierda a derecha, bajando fila por fila. Es el comportamiento natural de la memoria en la mayoría de los casos.

```
// El bucle externo maneja las filas
for (int f = 0; f < matrizDatos.GetLength(0); f++)
{
    // El bucle interno maneja las columnas
    for (int c = 0; c < matrizDatos.GetLength(1); c++)
    {
        Console.Write(matrizDatos[f, c] + " ");
    }

    Console.WriteLine(); // Salto de línea al terminar cada fila
}
```

Recorrido Columna - Fila (Por columnas)

Se procesa la matriz verticalmente: se lee toda la primera columna de arriba a abajo, luego la segunda, y así sucesivamente.

```
// El bucle externo ahora maneja las columnas
for (int c = 0; c < matrizDatos.GetLength(1); c++)
{
    // El bucle interno maneja las filas
    for (int f = 0; f < matrizDatos.GetLength(0); f++)
    {
```

```
        Console.Write(matrizDatos[f, c] + " ");  
  
    }  
  
    Console.WriteLine();  
  
}
```

Matrices Especiales:

Las matrices cuadradas tienen el mismo número de filas y columnas. En ellas se identifican zonas geométricas importantes.

Matriz Diagonal y la Diagonal Principal

La **diagonal principal** está formada por los elementos donde el índice de la fila es igual al de la columna. Una *matriz diagonal* es aquella donde todos los elementos fuera de esta línea son cero.

```
// Operación típica: Sumar la diagonal principal  
  
int sumaDiagonal = 0;  
  
int n = matrizCuadrada.GetLength(0); // Al ser cuadrada, filas == columnas  
  
for (int i = 0; i < n; i++)  
{  
  
    sumaDiagonal += matrizCuadrada[i, i]; // Solo requiere un único bucle  
  
}
```

Matrices Triangulares

- **Triangular Superior:** Todos los elementos **debajo** de la diagonal principal son cero. $f > c \Rightarrow \text{valor } 0$.
- **Triangular Inferior:** Todos los elementos **encima** de la diagonal principal son cero. Se cumple que $f < c \Rightarrow \text{valor } 0$.

```
// Ejemplo: Verificar si una matriz es Triangular Inferior

bool esTriangularInferior = true;

int tam = matrizCuadrada.GetLength(0);

for (int f = 0; f < tam; f++)
{
    for (int c = 0; c < tam; c++)
    {
        // Si está por encima de la diagonal (f < c) y no es cero, ya no es
        // triangular inferior

        if (f < c && matrizCuadrada[f, c] != 0)
        {
            esTriangularInferior = false;
            break;
        }
    }
}
```

Operaciones Típicas con Matrices:

Suma de dos matrices

Para sumar dos matrices A y B, ambas deben tener las mismas dimensiones. El resultado es una nueva matriz C donde: $C[F,C] = A[F,C] + B[F,C]$

```
int[,] A = { {1, 2}, {3, 4} };

int[,] B = { {5, 6}, {7, 8} };

int[,] C = new int[2, 2];

for (int f = 0; f < 2; f++)
{
    for (int c = 0; c < 2; c++)
    {
        C[f, c] = A[f, c] + B[f, c];
    }
}
```



Matriz Transpuesta

La transposición consiste en transformar las filas en columnas y las columnas en filas
(intercambiar `matriz[F,C]` por `matriz [C,F]`)

```
int filas = matrizOriginal.GetLength(0);

int columnas = matrizOriginal.GetLength(1);

int[,] transpuesta = new int[columnas, filas]; // Dimensiones invertidas


for (int f = 0; f < filas; f++)

{

    for (int c = 0; c < columnas; c++)

    {

        transpuesta[c, f] = matrizOriginal[f, c];

    }

}
```



